

Trabalho Prático 1: Estrutura de dados

Gabriel de Araújo Costa Gomes
2022102872

Departamento de Ciência da Computação; Universidade Federal de Minas Gerais
Belo Horizonte, Minas Gerais, Brasil; 10 de dezembro de 2024
gacg2004@ufmg.br

1 Introdução

Este trabalho aborda a análise de algoritmos de ordenação no contexto de localidade de referência e desempenho computacional. O problema tratado é organizar registros de dados utilizando algoritmos de ordenação indireta, considerando múltiplas chaves e mensurando o impacto no desempenho e na eficiência de memória. A solução emprega algoritmos clássicos de ordenação— como QuickSort, BubbleSort, e InsertionSort— e análise experimental utilizando a biblioteca `memlog` e `time` para monitorar o desempenho.

2 Método

A solução foi implementada em C, utilizando estruturas de dados para representar registros e índices. O processo foi dividido em várias etapas, com ênfase na ordenação indireta e no uso de diferentes algoritmos de ordenação. A seguir, detalharemos a organização dos arquivos, a configuração da máquina utilizada, o formato dos dados de entrada e saída, e as técnicas de ordenação aplicadas.

2.1 Organização dos Arquivos

A organização dos arquivos do projeto segue a seguinte estrutura:

```
\TP
\src
    main.c
    memlog.c
    ordInd.c
    sorts.c
\bin      # Arquivo executável
\obj      # Arquivos .o (objetos)
\include
    memlog.h
    odrInd.h
Makefile
```

- **Arquivo `main.c`:** Contém a função principal, onde o fluxo de execução é controlado, incluindo a leitura do arquivo de entrada, a criação dos índices, a ordenação dos registros e a impressão dos resultados.
- **Arquivo `ordInd.c`:** Contém as implementações das funções relacionadas à estrutura de dados, como a criação e destruição da estrutura, a manipulação dos índices e registros, e a ordenação dos dados.
- **Arquivo `sorts.c`:** Contém as implementações dos algoritmos de ordenação.
- **Arquivo `memlog.c`:** Implementa funções para capturar a localidade de referência e o tempo de execução utilizando a biblioteca `memlog`.
- **Arquivos de Entrada e Saída:** O formato de entrada é um arquivo CSV contendo os registros a serem processados, com campos separados por vírgulas. A saída consiste nos registros ordenados de acordo com os diferentes algoritmos de ordenação, com a exibição do cabeçalho original e os dados processados.

2.2 Ambiente de Execução

Os testes e experimentos foram realizados em uma máquina com as seguintes características:

- **Sistema Operacional:** Ubuntu 20.04 LTS.
- **Processador:** Intel Core™ i3-9100.
- **Memória RAM:** 4 GB.
- **Compilador:** GCC 10.3.0 com as opções `-Wall -g -fsanitize=address`.

2.3 Formato de Entrada e Saída

Entrada: O programa recebe como entrada um arquivo CSV contendo os dados a serem processados. O formato dos registros é o seguinte:

- As primeiras 6 linhas do arquivo contém, respectivamente, o cabeçalho com a quantidade de atributos, os nomes dos atributos e a quantidade de registros no arquivo.
- As linhas subsequentes contém os registros a serem ordenados.

Saída: O programa gera como saída os registros ordenados por cada atributo, utilizando todos os três algoritmos de ordenação implementados no código. Além disso, durante a análise experimental, o tempo de execução de cada etapa (carregamento, criação de índices, ordenação e impressão) foi exibido no terminal, e os acessos de memória foram registrados em um arquivo de log.

2.4 Ordenação Indireta

A ordenação indireta é um processo no qual os dados são ordenados por meio de índices, sem a necessidade de alterar diretamente os registros. Em vez de ordenar os registros diretamente, criamos índices que apontam para os registros e ordenamos esses índices. Isso permite manter a integridade dos dados enquanto organiza as referências a eles. No nosso caso, a estrutura de dados contém dois tipos principais:

- **Registro:** Contém os dados que serão manipulados.
- **ÍndiceIndireto:** Contém referências (índices) para os registros, que são ordenadas durante o processo de ordenação.

A ordenação dos índices, portanto, resulta na ordenação dos dados referenciados. O uso de ordenação indireta é vantajoso quando se deseja preservar a ordem original dos dados ou quando há uma necessidade de múltiplas ordenações por diferentes atributos.

2.5 Algoritmos de Ordenação

Foram implementados três algoritmos clássicos de ordenação, que foram aplicados aos índices para ordenar os dados:

- **QuickSort:** Um algoritmo de ordenação eficiente com complexidade média de $O(n \log n)$, mas com pior caso de $O(n^2)$. A implementação utiliza o método de divisão e conquista para ordenar os índices.
- **BubbleSort:** Um algoritmo simples, porém ineficiente em casos de grande volume de dados, com complexidade $O(n^2)$. Ele compara e troca os elementos adjacentes até que a lista esteja ordenada.
- **InsertionSort:** Um algoritmo que insere cada elemento na posição correta de uma lista já parcialmente ordenada, com complexidade $O(n^2)$ no pior caso, mas $O(n)$ no melhor caso.

Cada algoritmo foi aplicado ao mesmo conjunto de dados e os resultados comparados em termos de tempo de execução e localidade de referência.

2.6 Análise Experimental

A análise experimental foi conduzida para avaliar tanto o desempenho computacional quanto a localidade de referência. Para medir a localidade de referência, utilizamos a biblioteca *memlog*, que registra os acessos de memória realizados durante a execução do programa. O desempenho foi analisado em termos de tempo de execução utilizando a biblioteca *time*, medindo o tempo total de execução e o tempo de cada uma das etapas (carregamento, criação de índices, ordenação, impressão e destruição).

Os gráficos gerados incluem:

- Mapa de Acesso: Relaciona os endereços de memória com os tipos de acesso (leitura/escrita).
- Histograma de Distância de Pilha: Exibe a frequência das distâncias de pilha entre os acessos consecutivos.
- Evolução da Distância de Pilha: Mostra como a distância de pilha evolui ao longo do tempo.
- Comparação de Tempos de Execução: Compara o tempo total de execução para os diferentes algoritmos em diferentes cargas de trabalho.

Esses dados foram coletados e analisados para obter uma visão detalhada do comportamento dos algoritmos e da eficiência da implementação.

3 Análise de Complexidade

Nesta seção, analisamos a complexidade de tempo e espaço das principais funções implementadas no projeto. As análises consideram os cenários de pior caso, melhor caso e caso médio, conforme aplicável.

3.1 Funções principais

3.1.1 CarregaArquivo

- Descrição: Lê registros e atributos de um arquivo e os armazena em memória.
- Complexidade de tempo: $O(n + m)$, onde n é o número de registros e m o número de atributos. Cada linha do arquivo é processada uma vez.
- Complexidade de espaço: $O(n)$, devido à alocação de memória para armazenar os registros e atributos.

3.1.2 CriaIndice

- Descrição: Cria um índice indireto para um atributo, associando registros aos índices.
- Complexidade de tempo: $O(n)$, onde n é o número de registros. Cada registro é processado uma vez para criar o índice.
- Complexidade de espaço: $O(n)$, para armazenar os índices indiretos.

3.1.3 OrdenaIndice

- Descrição: Ordena o índice de registros com base em um atributo, utilizando diferentes algoritmos de ordenação.
- Complexidade de tempo:
 - QuickSort: $O(n \log n)$ no caso médio e melhor caso; $O(n^2)$ no pior caso.
 - BubbleSort: $O(n^2)$ em todos os casos.
 - InsertionSort: $O(n^2)$ no caso médio e pior caso; $O(n)$ no melhor caso (vetor já ordenado).
- Complexidade de espaço: $O(1)$, pois a ordenação ocorre in-place nos casos do BubbleSort e do InsertionSort. Já no caso do QuickSort a complexidade é $O(n)$.

3.1.4 ImprimeOrdenadoIndice

- Descrição: Exibe os registros de acordo com a ordem dos índices.
- Complexidade de tempo: $O(n)$, onde n é o número de registros, já que cada registro é acessado uma vez.
- Complexidade de espaço: $O(1)$, pois nenhum espaço adicional significativo é alocado.

3.1.5 Destroi

- Descrição: Libera toda a memória alocada para os registros, atributos e índices.
- Complexidade de tempo: $O(n + m)$, onde n é o número de registros e m o número de atributos.
- Complexidade de espaço: $O(1)$, já que apenas a memória previamente alocada é liberada.

3.2 Complexidade geral

O tempo total de execução do programa é dominado pelas seguintes operações:

- Leitura e carregamento: $O(n + m)$
- Criação de índices: $O(n)$ para cada índice.
- Ordenação: $O(n \log n)$ no caso médio para QuickSort.
- Impressão: $O(n)$ para cada índice.

O espaço utilizado é proporcional ao número de registros e atributos, mantendo-se dentro de $O(n + m)$, onde n é o número de registros e m o número de atributos.

4 Estratégias de Robustez

A robustez do código foi garantida por meio de diversas estratégias de programação defensiva e mecanismos para tratar falhas e inconsistências em tempo de execução. Essas estratégias foram aplicadas para assegurar que o programa pudesse lidar adequadamente com entradas inesperadas, erros de alocação de memória e condições limite, minimizando a possibilidade de comportamento indefinido ou falhas críticas.

4.1 Validação de entrada

Antes de processar qualquer dado, o programa valida os argumentos fornecidos pela linha de comando. Por exemplo:

- Caso o nome do arquivo não seja fornecido, uma mensagem de erro é exibida, e o programa termina sua execução imediatamente.
- No carregamento do arquivo (**CarregaArquivo**), verificações são realizadas para assegurar que o arquivo existe e está acessível. Caso contrário, uma mensagem de erro específica é emitida.
- Durante a leitura do arquivo, cada linha é analisada para verificar sua conformidade com o formato esperado. Linhas inválidas ou com campos ausentes são ignoradas, e uma mensagem de aviso é exibida.

4.2 Gerenciamento de memória

O código emprega boas práticas de gerenciamento de memória para evitar vazamentos e acessos inválidos:

- A função **safe_realloc** garante que erros de alocação de memória sejam tratados adequadamente. Caso a realocação falhe, o programa libera a memória previamente alocada e termina sua execução, evitando inconsistências.
- Antes de liberar estruturas complexas, como índices ou atributos, é feita uma verificação para assegurar que os ponteiros não sejam NULL, evitando tentativas de liberar memória não alocada.
- A função **Destroi** é responsável por liberar todos os recursos alocados, garantindo que não haja vazamentos de memória ao final da execução.

4.3 Tratamento de atributos e registros

Durante o processamento de atributos e registros, as seguintes verificações são realizadas:

- As funções `NomeAtributo` e `NumAtributos` verificam a validade do ponteiro para a estrutura antes de acessar os atributos, retornando um código de erro em caso de falha.
- Ao criar índices (`CriaIndice`), a memória anteriormente alocada para o índice é liberada, evitando vazamentos. Além disso, verificações garantem que índices inválidos não sejam processados.

4.4 Tolerância a falhas na ordenação

Na função `OrdenaIndice`, mecanismos garantem que somente atributos válidos sejam ordenados: Caso o atributo especificado não exista ou seja desconhecido, o programa exibe uma mensagem de erro informativa e interrompe a ordenação para aquele atributo.

4.5 Mensagens de depuração

Para facilitar a identificação de problemas, o programa inclui mensagens de depuração em pontos críticos:

- Cada etapa do processamento exibe mensagens detalhadas sobre o progresso, como carregamento de arquivos, criação de índices, ordenação e destruição da estrutura.
- Mensagens de erro são específicas e indicam exatamente onde ocorreu a falha, facilitando o diagnóstico e a correção.

4.6 Mecanismos de monitoramento

O uso da biblioteca `memlog` permite o monitoramento de acessos à memória, fornecendo uma camada adicional de análise e robustez. Isso ajuda a identificar padrões anômalos de acesso ou gargalos de desempenho associados à localidade de referência.

4.7 Manutenibilidade e extensibilidade

A modularidade do código, com funções bem definidas e separação de responsabilidades, contribui para a robustez do sistema. Alterações ou extensões podem ser realizadas com impacto mínimo, uma vez que cada função opera de maneira independente dentro de sua responsabilidade.

Com essas estratégias, o programa é capaz de operar de maneira segura, eficiente e resistente a falhas, mesmo em cenários adversos ou com entradas inesperadas.

5 Análise Experimental

Os experimentos foram realizados utilizando cargas de trabalho com diferentes tamanhos de entrada. Os resultados incluem:

5.1 Mapa de Acesso

Os mapas de acesso fornecem uma visão detalhada dos locais de memória acessados durante a execução do programa, com cada ponto representando uma operação de leitura ou escrita. Ao comparar as cargas de trabalho 1000/1000 (1000 registros e 1000 caracteres do ultimo atributos) e 5000/5000 (5000 registros e 5000 caracteres do ultimo atributos) , observa-se que, em ambos os casos, os acessos de memória são amplamente distribuídos. O padrão de dispersão dos pontos indica que os dados estão sendo acessados de maneira não contígua, resultando em distâncias de pilha maiores, o que pode levar a um custo adicional de acesso à memória.

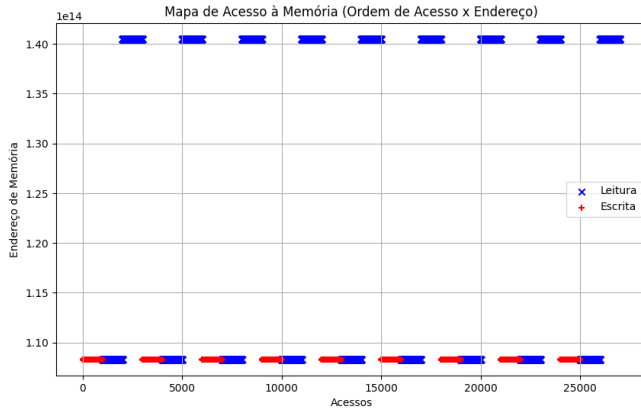


Figura 1: Mapa de acesso para carga r1000/p1000

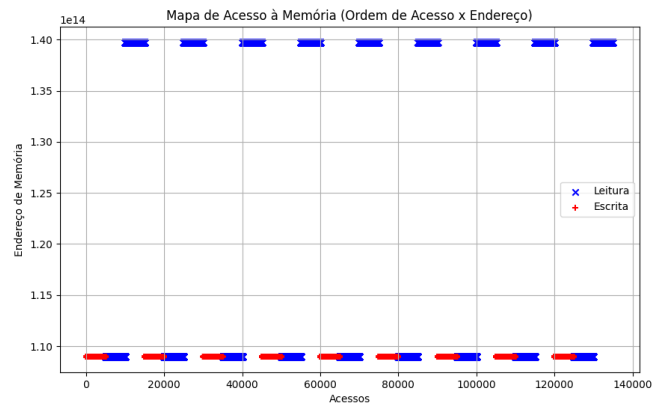


Figura 2: Mapa de acesso para carga r5000/p5000

5.2 Histograma de Distância de Pilha

Os histogramas revelam que as distâncias de pilha são grandes, indicando acessos de memória dispersos e, possivelmente, uma baixa localidade de referência. Isso significa que os dados não estão sendo acessados de maneira contígua na memória, o que pode resultar em maior custo de acesso, pois o processador precisa acessar diferentes áreas de memória em vez de aproveitar o cache de forma eficiente. Isso acontece mais notavelmente com o teste feito com a pilha de carga maior (note que o eixo y é muito maior).

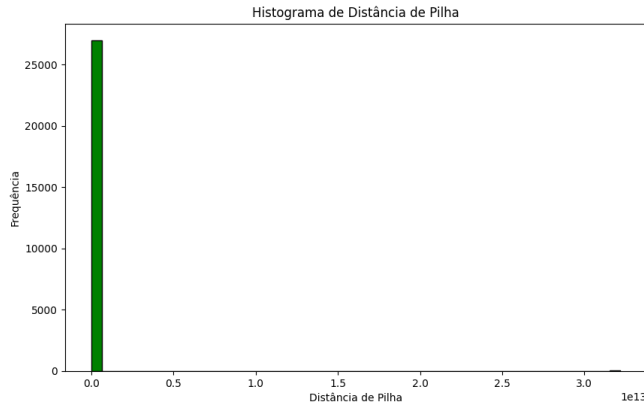


Figura 3: Distância de pilha para carga r1000/p1000

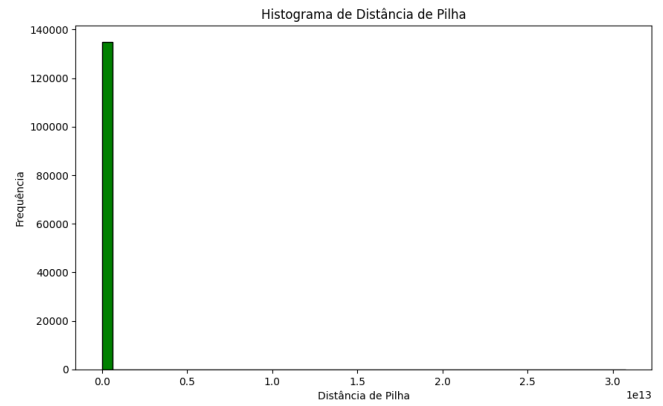


Figura 4: Distância de pilha para carga r5000/p5000

5.3 Evolução da Distância de Pilha

A evolução temporal demonstra que, embora existam picos de distância maiores em algumas operações (como ordenação), os acessos se mantêm relativamente próximos na maior parte do tempo.

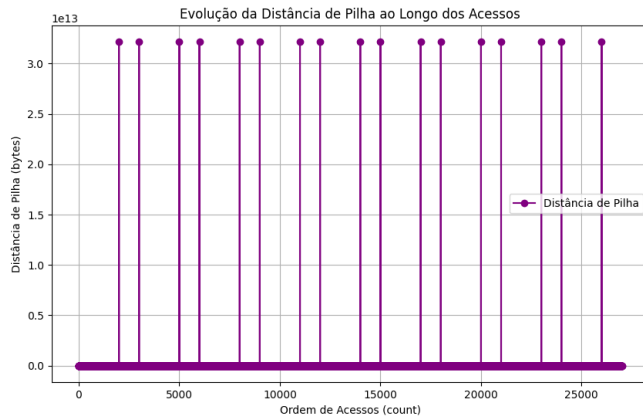


Figura 5: Evolução da Distância de pilha para carga r1000/p1000

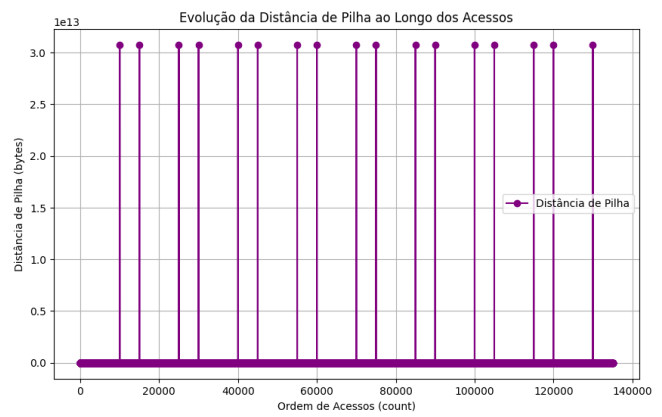


Figura 6: Evolução da Distância de pilha para carga r5000/p5000

5.4 Comparação de Tempos de Execução

5.4.1 Tempo total

O tempo total de execução do programa é dominado por várias operações principais. A leitura e o carregamento do arquivo possuem uma complexidade linear em relação ao número de registros e atributos, ou seja, $O(n + m)$, onde n é o número de registros e m é o número de atributos. O gráfico a seguir evidencia isso.

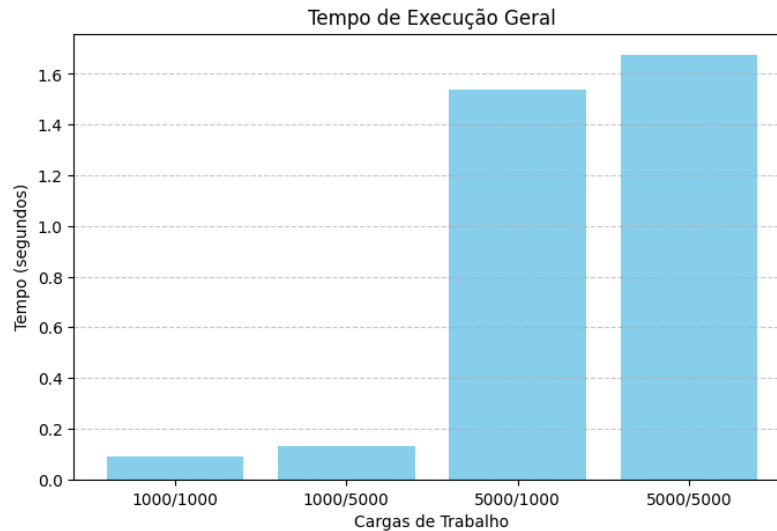


Figura 7: Tempo de Execução Geral

5.4.2 Criação

O tempo de criação da estrutura foi diretamente influenciado pelo número de registros e pela complexidade da alocação de memória para os índices. Em cargas menores, como a de 1000 registros, a criação foi relativamente rápida, mas à medida que o número de registros e atributos aumentou, o tempo necessário para alocar e inicializar os dados também aumentou. Isso ocorreu principalmente devido ao aumento do número de índices que precisavam ser alocados e inicializados.

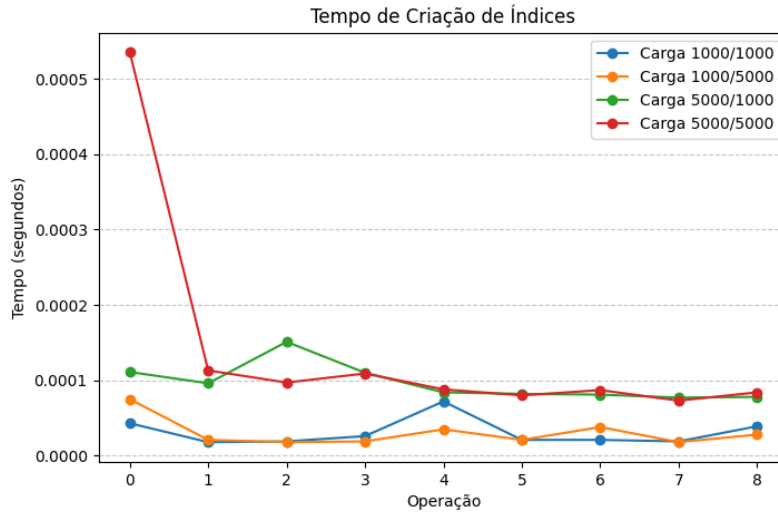


Figura 8: Tempo de Execução de Criação

5.4.3 Carregamento

O tempo de carregamento dos dados também apresentou variação em função da carga de trabalho. Inicialmente, para cargas pequenas, o tempo de carregamento foi baixo, já que o arquivo de entrada é relativamente pequeno e os registros são lidos de maneira direta. No entanto, para cargas maiores, o tempo de carregamento aumentou consideravelmente, dado que a quantidade de dados a ser lida e processada aumentou, o que resultou em mais operações de leitura do disco e maior demanda de memória. Nesse gráfico, podemos notar que a quantidade de caracteres— ou seja, o tamanho do registro— influenciou mais no resultado do que a quantidade de registros (veja bem as cargas de trabalho 1000/5000 e 5000/1000).

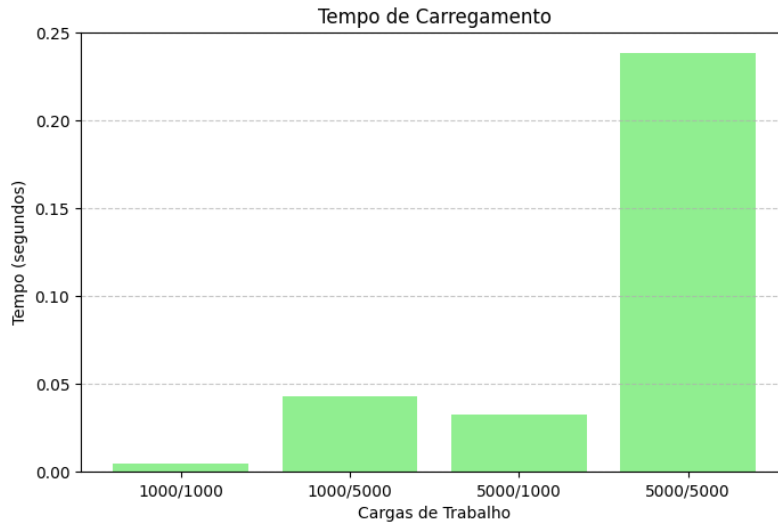


Figura 9: Tempo de Execução de Carregamento

5.4.4 Ordenação

O QuickSort apresentou o melhor desempenho em cargas maiores devido à sua complexidade assintótica $O(n \log n)$. O BubbleSort foi significativamente mais lento, como esperado devido à sua complexidade $O(n^2)$. O InsertionSort teve desempenho intermediário, sendo muito efetivo em cargas pequenas, mas superado pelo QuickSort em cargas maiores.

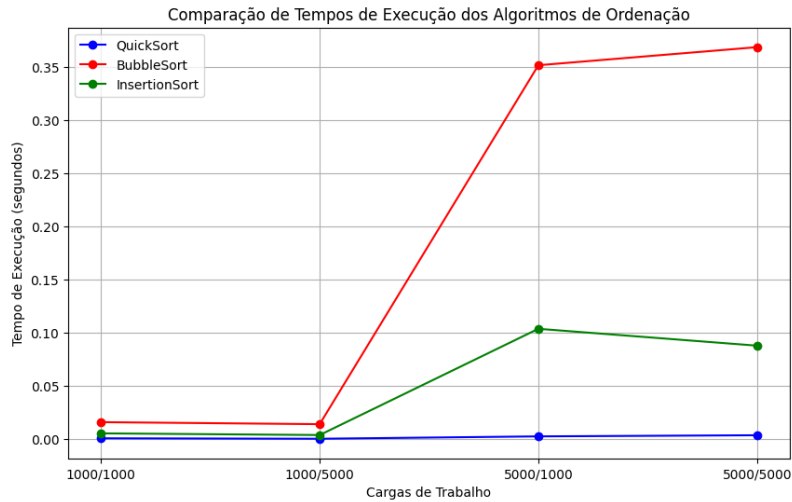


Figura 10: Comparação de tempos de Execução de Ordenação

5.4.5 Impressão

A impressão da estrutura, que envolve a ordenação e exibição dos registros, também apresentou um aumento de tempo conforme a carga de trabalho aumentava. Esse aumento foi especialmente notável para as cargas com maior número de registros e caracteres, devido ao custo de ordenação e à maior quantidade de dados sendo processados. A impressão é uma etapa intensiva de I/O e envolve o acesso sequencial a grandes quantidades de dados, o que pode resultar em tempos de execução mais longos quando a carga de trabalho cresce.

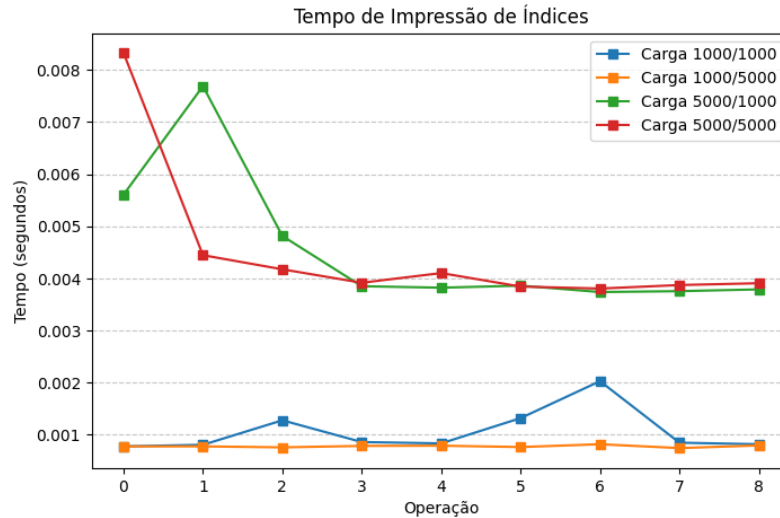


Figura 11: Tempo de Execução de Impressão

5.4.6 Destruição

A destruição da estrutura, que envolve a liberação da memória alocada, também mostrou um aumento nos tempos de execução à medida que o número de registros e índices crescia. Embora a destruição seja uma operação relativamente simples em termos de lógica, o aumento no volume de dados e na quantidade de memória alocada causou um tempo maior de execução para liberar todos os recursos corretamente. Note que o tamanho do registro não tem muita influência no tempo gasto.

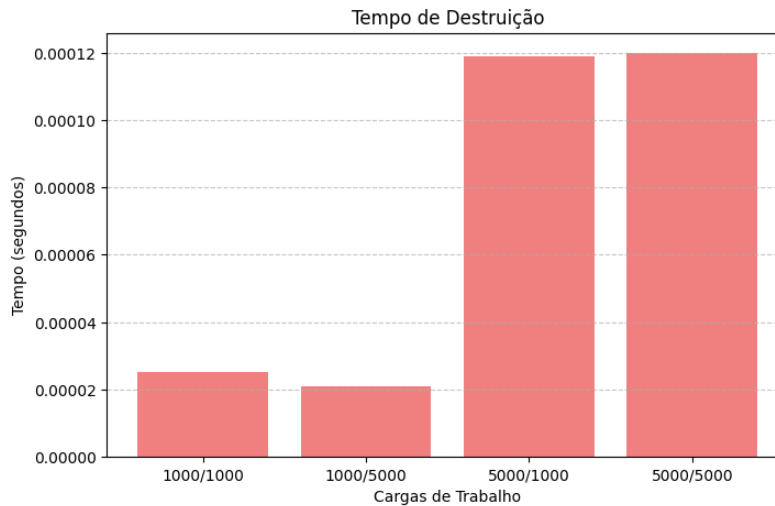


Figura 12: Tempo de Execução de Destruição

6 Conclusões

Através da análise experimental, foi possível observar o impacto do aumento da carga de trabalho no desempenho dos algoritmos de ordenação, bem como no comportamento da localidade de referência da memória.

Os resultados indicaram que, à medida que o número de registros e o tamanho deles aumentava, o tempo de execução e a distância de pilha também cresciam, refletindo a complexidade crescente do problema. Além disso, os gráficos de mapa de acesso e evolução de distância de pilha mostraram como o acesso à memória se torna menos eficiente à medida que o volume de dados cresce, o que sugere a necessidade de otimização tanto no uso de memória quanto no design dos algoritmos de ordenação.

O trabalho mostrou ainda a importância de estratégias de robustez, como a verificação de falhas de alocação e a utilização de um log de memória para avaliar a localidade de referência. Essas estratégias garantiram que o programa fosse capaz de lidar com grandes volumes de dados de maneira eficiente e segura.

Em suma, a implementação foi bem-sucedida na tarefa de ordenar dados de forma eficiente, mas os resultados também destacaram áreas para melhoria, especialmente no que diz respeito à otimização do uso de memória e à redução dos tempos de acesso à memória em cenários de alta carga.

7 Bibliografia

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
2. Memlog: Ferramenta de Monitoramento de Localidade de Referência. Slides disponíveis em: <https://docs.google.com/presentation/d/1cruITvxDEkzXkHtrgqduG73FSyXEOSKmRPwUdRcoyI4/edit#slide=id.p1>. Último acesso em 10 de dezembro de 2024.
3. Aula 05 - Ordenação: Bolha, Inserção e Seleção. Slides disponíveis em: <https://docs.google.com/presentation/d/1h3xBjbPZd2Cw1DVZIpUTM95zPtXZbh0/edit#slide=id.p114>. Último acesso em 10 de dezembro de 2024.
4. Aula 07 - QuickSort. Slides disponíveis em: https://docs.google.com/presentation/d/1xIXZE59u-oF3Trk3kPwHCRg-9/edit#slide=id.g229ad1436ef_2_293. Último acesso em 10 de dezembro de 2024.